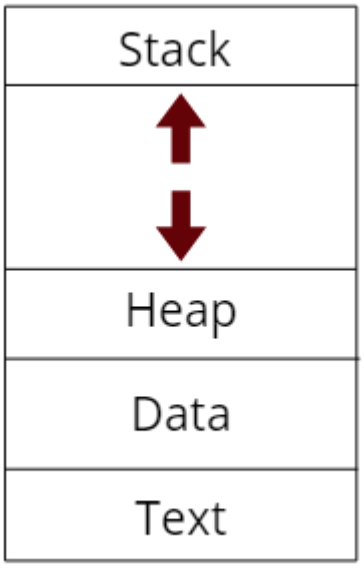
Process Management in OS

- A program does nothing unless its instructions are executed by CPU.
- A program in execution is called a process.
- In order to accomplish its task, process needs the computer resources.
- There may exist more than one process in the system which may require the same resource at the same time. Therefore, the operating system has to manage all the processes and the resources in a convenient and efficient way.
- Some resources may need to be executed by one process at one time to maintain the consistency otherwise the system can become inconsistent, and deadlock may occur.
  - A deadlock occurs when two or more processes are unable to proceed because they are waiting for each other to release resources.
- The operating system is responsible for the following activities in connection with Process Management:

0. Scheduling process and threads on the CPU.
1. Creating and deleting both user and system processes.
2. Suspending and resuming processes.
3. Providing mechanisms for process synchronization.
4. Providing mechanisms for process communication.

What is process?

- A process is a program in execution including the current values of the program counter, registers and variables.
- The difference between a process and a program is that the program is the group of instruction where as the process is the activity. Or we can say process is an active state of program.
- We write our computer programs in a text file and when we execute this program, it becomes a process which performs all the tasks mentioned in the program.
- **When a program is loaded into the memory and it becomes a process, it can be divided into four sections: stack, heap, text and data.**
- A process generally also includes the process stack, which contains temporary data (such as function parameters, return addresses, and local variables), and a data section, which contains global variables.

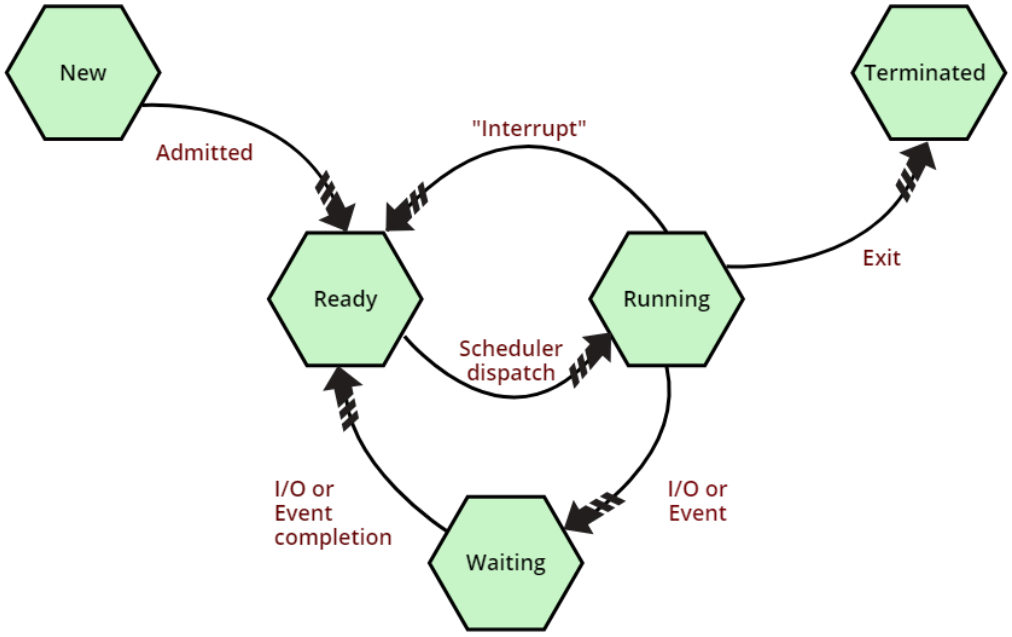


Main memory

- Stack: The process stack contains the temporary data such as method/function, parameters, return address and local variables.
- Heap: This is dynamically allocated memory to a process during its runtime.
- Text: Program counters (It stores the list of address of next instructions that has to be executed after the current process) & content of processor registers.
- Data: This section contains the all global and static variables.

Process States

- When a process executes, it passes through different states.
- These stages may differ in different operating systems.
- In general, a process can have one of the following five states at a time.



Process Life

- **New:** This is the initial state when a process is first started/created.
- **Ready:** The process are waiting to have the processor allocated to them by the operating system so that they can run.
  - Process may come into this state after leaving the start state or while running but be interrupted by the scheduler to assign CPU to some other process.
- **Running**: After *Ready* state, the process state is set to running and the processor execute its instruction.
- **Waiting:** Process moves into the waiting state if it needs to wait for a resource, such as waiting for user input, or waiting for a file to become available.
- **Terminated:** Once the process finishes its execution or its is terminated by the operating system, it is moved to the terminated state where it waits to be removed from main memory.

Attributes of a process

- The attributes of the process are used by the operating system to create the **process control block (PCB) for each of them.**
- This is also called context of the process. Attributes which are stored in the PCB are described below.

1. **Process ID**: When a process is created, a unique id is assigned to the process which is used for unique identification of the process in the system.
2. **Program counter**: A program counter stores the address of the last instruction of the process on which the process was suspended. The CPU uses this address when the execution of this process is resumed.
3. **Process State**: The process, from its creation to the completion, goes through various states which are new, ready, running and waiting.
4. **Priority**: Every process has its own priority. The process with the highest priority among the process gets the CPU first. This is also stored on the process control block.
5. **General Purpose Registers**: Every process has its own set of registers which are used to hold the data which is generated during the execution of the process.
6. **List of open files**: During the execution, every process uses some files which need to be present in the main memory. OS also maintains a list of open files in the PCB.
7. **List of open devices**: OS also maintain the list of all open devices which are used during the execution of the process.

Process ID
Process Counter
Process State
Priority
General Purpose Registers
List of open files
List of open devices

Process Scheduling in Operating System

Definition

The process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process on the basis of a particular strategy.

Process scheduling is an essential part of a Multiprogramming operating systems. Such operating systems allow more than one process to be loaded into the executable memory at a time and the loaded process shares the CPU using time multiplexing.

Categories of Scheduling

There are two categories of scheduling:

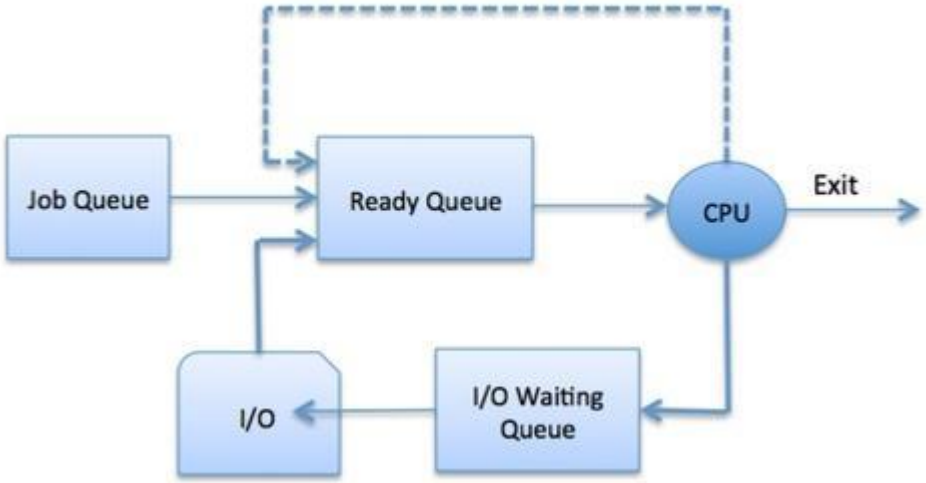
1. **Non-preemptive:** Here the resource can't be taken from a process until the process completes execution. The switching of resources occurs when the running process terminates and moves to a waiting state.
2. **Preemptive:** Here the OS allocates the resources to a process for a fixed amount of time. During resource allocation, the process switches from running state to ready state or from waiting state to ready state. This switching occurs as the CPU may give priority to other processes and replace the process with higher priority with the running process.

Process Scheduling Queues

The OS maintains all Process Control Blocks (PCBs) in Process Scheduling Queues. The OS maintains a separate queue for each of the process states and PCBs of all processes in the same execution state are placed in the same queue. When the state of a process is changed, its PCB is unlinked from its current queue and moved to its new state queue.

The Operating System maintains the following important process scheduling queues –

- **Job queue** – This queue keeps all the processes in the system.
- **Ready queue** – This queue keeps a set of all processes residing in main memory, ready and waiting to execute. A new process is always put in this queue.
- **Device queues** – The processes which are blocked due to unavailability of an I/O device constitute this queue.



The OS can use different policies to manage each queue (FIFO, Round Robin, Priority, etc.). The OS scheduler determines how to move processes between the ready and run queues which can only have one entry per processor core on the system; in the above diagram, it has been merged with the CPU.

Two-State Process Model

Two-state process model refers to running and non-running states which are described below –

S.N.	State & Description
1	**Running** When a new process is created, it enters into the system as in the running state.
2	**Not Running** Processes that are not running are kept in queue, waiting for their turn to execute. Each entry in the queue is a pointer to a particular process. Queue is implemented by using linked list. Use of dispatcher is as follows. When a process is interrupted, that process is transferred in

the waiting queue. If the process has completed or aborted, the process is discarded. In either case, the dispatcher then selects a process from the queue to execute.

Schedulers

Schedulers are special system software which handle process scheduling in various ways. Their main task is to select the jobs to be submitted into the system and to decide which process to run. Schedulers are of three types –

- Long-Term Scheduler
- Short-Term Scheduler
- Medium-Term Scheduler

Long Term Scheduler

It is also called a **job scheduler**. A long-term scheduler determines which programs are admitted to the system for processing. It selects processes from the queue and loads them into memory for execution. Process loads into the memory for CPU scheduling.

The primary objective of the job scheduler is to provide a balanced mix of jobs, such as I/O bound and processor bound. It also controls the degree of multiprogramming. If the degree of multiprogramming is stable, then the average rate of process creation must be equal to the average departure rate of processes leaving the system.

On some systems, the long-term scheduler may not be available or minimal. Time-sharing operating systems have no long term scheduler. When a process changes the state from new to ready, then there is use of long-term scheduler.

Short Term Scheduler

It is also called as **CPU scheduler**. Its main objective is to increase system performance in accordance with the chosen set of criteria. It is the change of ready state to running state of the process. CPU scheduler selects a process among the processes that are ready to execute and allocates CPU to one of them.

Short-term schedulers, also known as dispatchers, make the decision of which process to execute next. Short-term schedulers are faster than long-term schedulers.

Medium Term Scheduler

Medium-term scheduling is a part of **swapping**. It removes the processes from the memory. It reduces the degree of multiprogramming. The medium-term scheduler is in-charge of handling the swapped out-processes.

A running process may become suspended if it makes an I/O request. A suspended processes cannot make any progress towards completion. In this condition, to remove the process from memory and make space for other processes, the suspended process is moved to the secondary storage. This process is called **swapping**, and the process is said to be swapped out or rolled out. Swapping may be necessary to improve the process mix.

Comparison among Scheduler

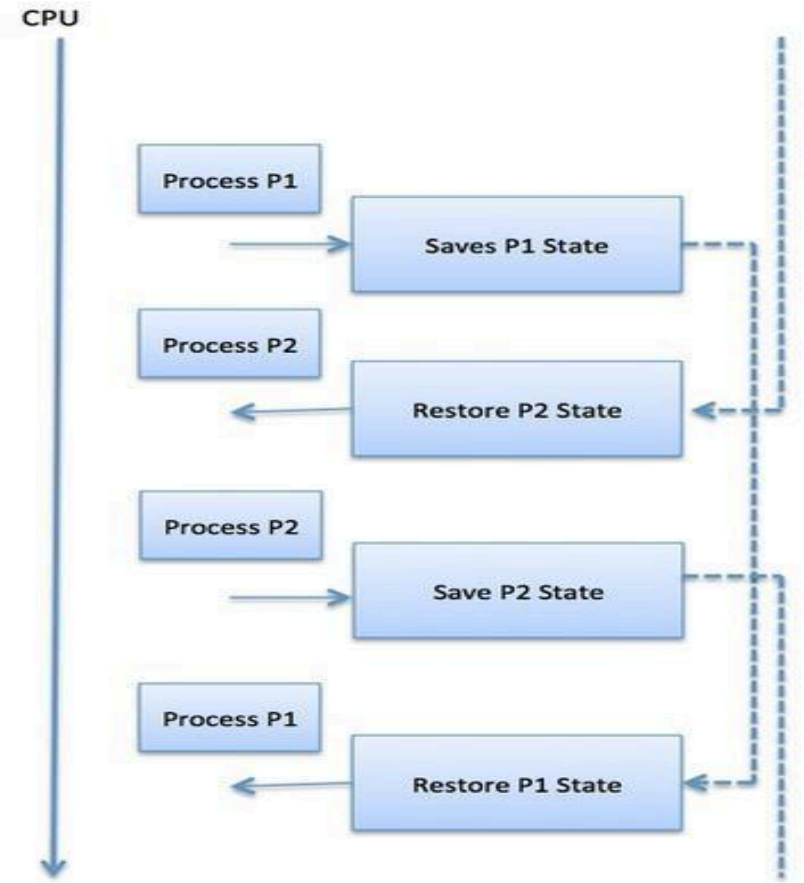
S.N.	Long-Term Scheduler	Short-Term Scheduler	Medium-Term Scheduler
1	It is a job scheduler	It is a CPU scheduler	It is a process swapping scheduler.
2	Speed is lesser than short term scheduler	Speed is fastest among other two	Speed is in between both short and long term scheduler.
3	It controls the degree of multiprogramming	It provides lesser control over degree of multiprogramming	It reduces the degree of multiprogramming.
4	It is almost absent or minimal in time sharing system	It is also minimal in time sharing system	It is a part of Time sharing systems.
5	It selects processes from pool and loads them into memory for execution	It selects those processes which are ready to execute	It can re-introduce the process into memory and execution can be continued.

Context Switching

A context switching is the mechanism to store and restore the state or context of a CPU in Process Control block so that a process execution can be resumed from the same point at a later time. Using this technique, a context switcher enables multiple processes to share a single CPU. Context switching is an essential part of a multitasking operating system features.

When the scheduler switches the CPU from executing one process to execute another, the state from the current running process is stored into

the process control block. After this, the state for the process to run next is loaded from its own PCB and used to set the PC, registers, etc. At that point, the second process can start executing.

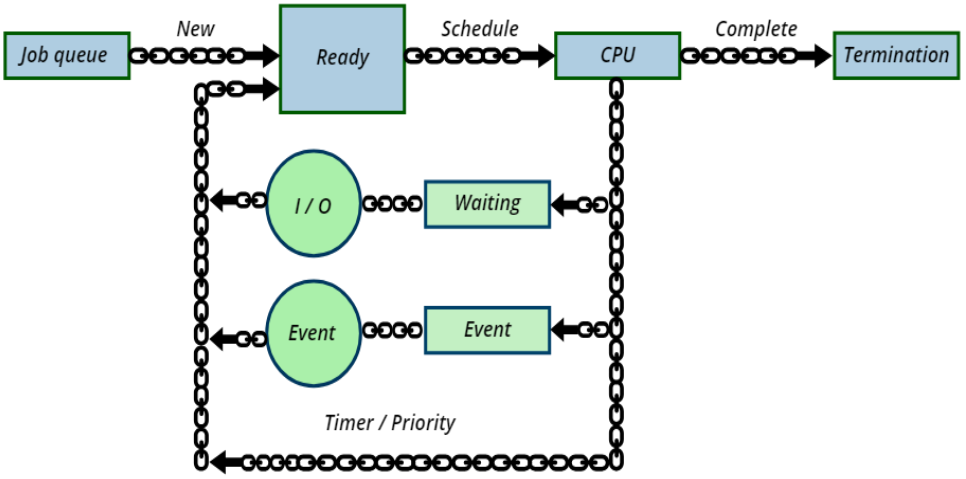


Context switches are computationally intensive since register and memory state must be saved and restored. To avoid the amount of context switching time, some hardware systems employ two or more sets of processor registers. When the process is switched, the following information is stored for later use.

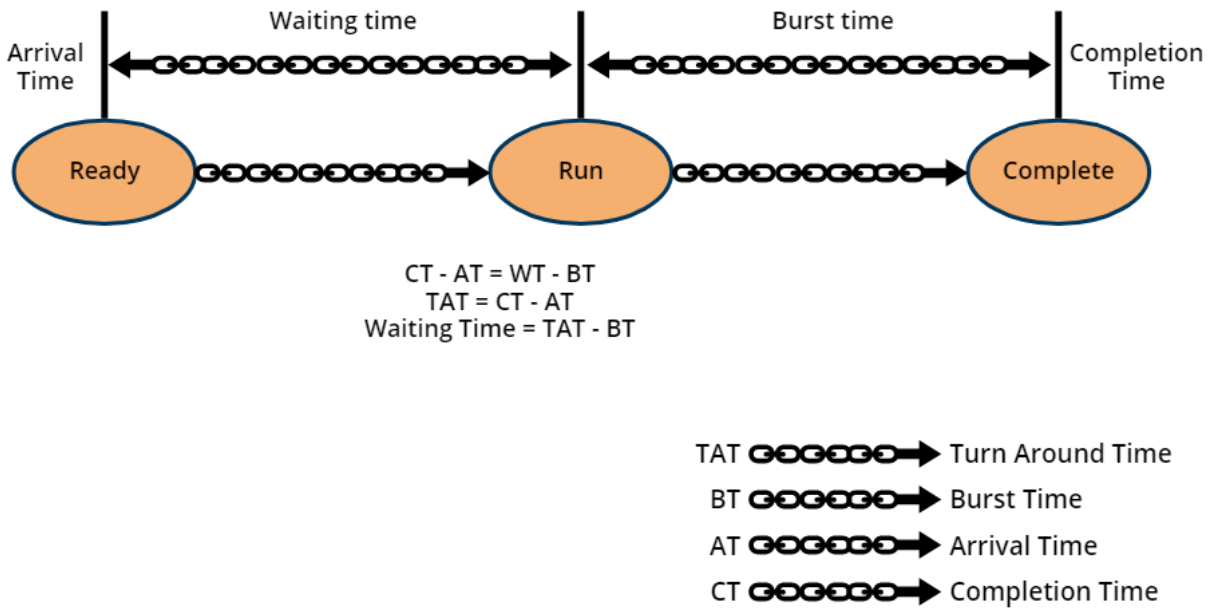
- Program Counter
- Scheduling information
- Base and limit register value
- Currently used register
- Changed State
- I/O State information
- Accounting information

Process Queues

- The operating system manages various types of queues for each of the process states.
- The PCB related to the process is also stored in the queue of the same state.
- If the process is moved from one state to another state then its PCB is also unlinked from the corresponding queue and added to the other state queue in which the transition is made.



1. **Job Queue**
   - In starting, all the process get stored in the job queue.
   - It is maintained in the secondary memory.
   - The long term scheduler (Job scheduler) picks some of the process and put them in the primary memory.
2. **Ready Queue**
   - Ready queue is maintained in primary memory.
   - The short term scheduler picks the job from the ready queue and dispatch of the CPU for the execution.
3. **Waiting Queue**
   - When the process needs some IO operation in order to complete its execution, OS Changes the state of the process from running to waiting.
   - The context (PCB) associated with the process gets stored on the waiting queue which will be used by the processor when the process finishes the IO.



Various Times related to Process:

1. **Arrival Time**
   - The time at which the process enters into the ready queue.
2. **Burst Time**
   - The total amount of time required by the CPU to execute the whole process is called the Burst Time.
   - This does not include the waiting time.
3. **Completion Time**
   - The time at which the process enters into the completion state or the time at which the process completes its execution.
4. **Turnaround time**
   - The total amount of time spent by the process from its arrival to its completion.
5. **Waiting Time**
   - The total amount of time for which the process waits for the CPU to be assigned.
6. **Response Time**
   - The difference between the arrival time and the time at which the process first gets the CPU.

CPU Scheduling

- **In the uniprogramming systems** like MS DOS, when a process waits for I/O operation to be done, the CPU remains idol.
  - Uniprogramming is a type of operating system in which only one program can be executed at a time.

- This is an overhead since it wastes the time and causes the problem of starvation.
  - "overhead" refers to the extra resources that are required by the system to perform tasks beyond what is required for the user's program. Overhead can include the time, memory, processing power, or other system resources that are needed to manage and run the operating system itself.
- However, in **multiprogramming systems**, the CPU doesn't remain idle during the waiting time of the process and it starts executing other processes.
- Operating system has to define which process the CPU will be given.
- **In multiprogramming systems**, the OS schedules the processes on the CPU to have the maximum utilization of it and this procedure is called **CPU scheduling**. The Operating system uses various scheduling algorithm to schedule the processes.
- This is a task of the short term schedular to schedule the CPU for the number of processes present in ready queue.
- Whenever the running process requests some I/O operation then the short term scheduler saves the current context of the process (also called PCB) and changes its state from running to waiting. During the time, process is in waiting state; the short term shceduler picks another process from the ready queue and assigns the CPU to this process. This procedure is called **context switching.**

What is saved in the Process Control Block?

- The OS maintains a process control block during the lifetime of the process.
- Process control block is a data structure used in operating system to store all data related information to the process.
- The process control block is deleted when the process is terminated or killed.
- There is the following information which is saved in the PCB and is changing with the state of the process.

1. Process ID → A unique identifier for the process.
2. Process State → The current state of the process (running, waiting, etc.)
3. Pointer → Pointers to the PCBs of the parent and child processes, and other relevant data structures.

4. Priority → The priority of the process in relation to other processes in the system.
5. Program Counter → The address of the next instruction to be executed.
6. CPU Registers → The values of the CPU registers for the process.
7. I/O status information → Information about the process's open files, pending I/O requests, and other I/O-related data.
8. Accounting Information → The amount of CPU time used, clock time since process creation, and other information used for process accounting.
9. etc.

Different criteria used for CPU scheduling in an operating system:

1. CPU utilization: This refers to the percentage of time that the CPU is being used. A scheduling algorithm that aims to maximize CPU utilization will try to keep the CPU busy as much as possible.
2. Throughput: This refers to the number of processes that are completed per unit time. A scheduling algorithm that aims to maximize throughput will try to maximize the number of processes that are completed in a given amount of time.
3. Turnaround Time: This is the amount of time it takes for a process to complete, from the moment it enters the ready queue until it finishes executing. A scheduling algorithm that aims to minimize turnaround time will try to minimize the time that processes spend waiting in the ready queue.
4. Waiting Time: This is the amount of time that a process spends waiting in the ready queue. A scheduling algorithm that aims to minimize waiting time will try to minimize the amount of time that processes spend waiting in the ready queue.
5. Response Time: This is the amount of time it takes for a process to start responding after a request has been made. A scheduling algorithm that aims to minimize response time will try to minimize the time that processes spend waiting in the ready queue before they start executing.

- There are various algorithms which are used by the OS to schedule the processes on the processor in an efficient way.

The purpose of a Scheduling algorithm

1. Maximum CPU utilization
2. Fare allocation of CPU

3. Maximum throughput
4. Minimum turnaround time
5. Minimum waiting time
6. Minimum response time

There are the following algorithms which can be used to schedule the process.

1. First Come First Serve
2. Shortest Job First
3. Round Robin

First Come First Server CPU Process Scheduling in OS

- This is the basic algorithm which every student must learn to understand all the basics of CPU Process Scheduling Algorithms.
- First Come First Serve paves the way for understanding of other algorithms.
- This algorithm may have many disadvantages, but these disadvantages created very new and efficient algorithms. So, we should learn First Come First Serve first.
- **Important Abbreviations:**

1. CPU → Central Processing Unit
2. FCFS → First Come First Serve
3. AT → Arrival Time
4. BT → Burst Time
5. WT → Waiting Time
6. TAT → Turn Around Time
7. CT → Completion Time
8. FIFO → First In First Out

First Come First Serve

- FCFS is the first algorithm of CPU process scheduling algorithm.
- FCFS allow the process to execute in linear manner.
- This means that whichever process enters the ready queue first is executed first.
- This shows that FCFS algorithm follows FIFO principle.
- The FCFS algorithm can be executed in Pre Emptive and Non Pre Emptive manner.

Preemptive Approach

- Here the OS allots the resources to a process for a predetermined period of time.
- The process transitions from running state to ready state or from waiting state to ready state during resource allocation.
- This switching happens because the CPU may assign other processes precedence and substitute the currently active process for the higher priority process.

Non Preemptive Approach

- Here the resource cannot be withdrawn from a process before the process has finished running.
- When a running process finishes and transitions to the waiting state, resources are switched.

Convoy Effect in FCFS

- Convoy effect is a phenomenon which occurs in the scheduling algorithm named FCFS.
- The convoy effect occurs when the FCFS Scheduling algorithm occurs in a way of non preemptive way.
- The non preemptive way means that if a process or job is started execution, then the operating system must complete is process or job.
- Until, the process is zero the next process does not start its execution.
- The definition of Non Preemptive scheduling in terms of OS means that the CPU will be completely dedicated till the end of the process started first and the new process is executed only after finishing of the older process.
- There may be a few cases, which might cause the CPU to allot a too much time. This is because in the FCFS sheduling algorithm non preemptive approach, the process are chose in serial oder. Due, to this shorter process behind the larger process takes too much time to complete its execution. Due, to this the WT, TAT, CT is very high.
- If the first process if large or completion time is too high, then convoy effect in the FCFS algorithm is occured.
- Let us assume that longer job takes infinite time to complete. Then, the remaining processes have to wait for the same infinite time. Due to this convoy effect created by the longer job the starvation of the waiting processes increases very rapidly. this is the biggest disadvantage of FCFS CPU process scheduling.

Characteristics of FCFS CPU Process Scheduling

1. Implementation is simple.
2. Does not cause any casualties while using.
3. It adopts a non pre-emptive and pre-emptive strategy.
4. It runs each procedure in the order that they are recieved.
5. Arrival time is used as a selection criterin for procedures.

Advantages of FCFS CPU Process Scheduling

1. In order to allocate processes, it uses the FIFO queue.
2. The FCFS CPU Scheduling Process is straight forward and easy to implement.
3. In the FCFS situation pre emptive scheduling, there is not chance of process starving.
4. As there is no consideration of process priority, it is an equitable algorithm.

Disadvantages of FCFS CPU Process Scheduling

- FCFS CPU Scheduling Algorithm has Long Waiting Time.
- FCFS CPU Scheduling favors CPU over Input or Output operations.
- In FCFS there is a chance of occurrence of Convoy Effect.
- Because FCFS is so straight forward, it often isn't very effective. Extended waiting periods go hand in hand with this. All other orders are left idle if the CPU is busy processing one time-consuming order.

Problems on the FCFS CPU Scheduling Algorithm

This is how the FCFS is solved in Non Pre Emptive Approach.

Shortest Job First (SJF) Scheduling

- SJF Scheduling algorithm, schedules the processes according to their burst time.
- In SJF scheduling, the process with the lowest burst time, among the list of available processes in the ready queue, is going to be scheduled next.
- However, it is very difficult to predict the burst time needed for a process hence this algorithm is very difficult to implement in the system.

Advantages of SJF

- Maximum throughput
- Minimum average waiting and turnaround time

Disadvantages of SJF

- May suffer with the problem of starvation.
- It is not implementable because the exact burst time for a process can't be known in advance.

Non-Preemptive shortest job first CPU Scheduling Algorithm example:

- **Advantages:**
  - SJF is better than the FCFS algorithm as it reduces the average waiting time.
  - SJF is genrally used for long term scheduling.
  - It is suitable for the jobs running in batches, where run times are already known.
  - SJF is probably optimal in terms of average turnaround time.
- **Disadvantages:**
  - SJF may cause very long turn-around times or starvation.
  - In SJF job completion timme must be known earlier, but sometimes it is hard to predict.
  - Sometimes, it is complicated to predict the length of the upcoming CPU request.
  - It leads to the starvation that does not reduce average turnaround time.

Preemptive SJF - Shortest Remaining Time First Scheduling Algorithm

- In SRTF schefuling algorithm, the process with the smallest amount of time remaining until completion is selected to execute.
- Since the currently executing process is the one with the shortest amount of time remaining by definition, and since that time should only reduce as execution progresses, process will always run until they complete or a new process is added that requires a smaller amount of time.

Examples to show working of Preemptive Shortest Job First CPU Scheduling Algorithm ↓

Round Robin Scheduling Algorithm

- In RR algorithm each process is assigned a fixed time slot in a cyclic way.
- It is basically the preemptive version of FCFS scheduling algorithm.
- RR CPU algorithm generally focuses on Time sharing technique.
- The period of time for which a process or job is allowed to run in a pre-emptive method is called time quantum.
- Each process present in the ready queue is assigned the CPU for that time quantum, if the execution of the process is completed during that time then the process will end else the process will go back to the waiting table and wait for its next turn to complete the execution.

Characteristics of RR:

- It is simle, easy to implement, and starvation-free as all process get fair share of CPU time.
- One of the most commonly used technique in CPU scheduling as a core.
- It is preempitve as process are assigned CPU only for a fixed slice of time at most.
- The disadvantages of it is more overhead of context switching.

Advantages of RR:

- There is fairness since every process gets equal share of CPU.
- The newly created process is added to end of ready queue.
- A round-robin scheduler generally employs time-sharing, giving each job a time slot or quantum.
- While performing a round-robin scheduling, a particular time quantum is allotted to different jobs.
- Each process get a chance to reschedule after a particular quantum time in this scheduling

Disadvantages of RR:

- There is Larger waiting time and Response time.
- There is Low throughput

- There is Context Switches
- Gantt chart seems to come too big (if quantum time is less for scheduling.
- Time consuming scheduling for small quantum

Consider the following table of arrival time and burst time for four processes P1, P2, P3, and P4 and given Time Quantum = 2. ↓

Multiple-Processor Scheduling

- In multiple-processor scheduling multiple CPU's are available and hence **load sharing** becomes possible.
- However multiple processor scheduling is more complex as compared to single processor scheduling.
- In multiple processor scheduling there are cases when the processors are identical i.e. HOMOGENEOUS, in terms of their functionality, we can use any processor available to run any process in the queue.

Approaches to Multiple-Processor Scheduling:

- One approach is when all the scheduling decisions and I/O processing are handled by a single processor which is called the **master server** and the other processors executes only the **user code**. This is simple and reduces the need of data sharing. This entire scenario is called **asymmetric multiprocessing.**
- A second approach uses **symmetric multiprocessing** where each processor is self scheduling. All processes may be in common ready queue or each processor may have its own private queue for ready processes. The scheduling proceeds further by having the scheduler for each processor examine the ready queue and select a process to execute.

Processor Affinity

- Processor affinity is the ability to bind a process or thread to a specific CPU or core in a multi-core system. This means that the process or thread will be scheduled to run exclusively on the chosen CPU or core, and will not be allowed to run on any other CPU or core in the system.
- The purpose of processor affinity is to improve the performance of a system by reducing the overhead of context switching between different CPUs or cores. By binding a process or thread to a specific CPU or core,

the operating system can reduce the time and resources needed to switch between different CPUs or cores, which can result in faster execution times and more efficient use of system resources.

- When a process runs on a specific processor there are certain effects on the cache memory. The data most recently accessed by the process populate the cache for the processor and as a result successive memory access by the process are often satisfied in the cache memory. Now if the process migrates to another processor, the contents of the cache memory must be invalidated for the first processor and the cache for the second processor must be repopulated. Because of the high cost of invalidating and repopulating caches, most of the SMP(symmetric multiprocessing) systems try to avoid migration of processes from one processor to another and try to keep a process running on the same processor. This is known as PROCESSOR AFFINITY.

There are two types of processor affinity:

1. **Soft Affinity:** When an operating system has a policy of attempting to keep a process running on the same processor but not guaranteeing it will do so, this situation is called soft affinity.
2. **Hard Affinity:** Hard affinity allows a process to specify a subset of processors on which it may run. Some systems such as Linux implements soft affinity but also provide some system calls like *sched_setaffinity()* that supports hard affinity.

Load Balancing

- Load balancing is a phenomena which keeps the workload evenly distributed accross all processors in a SMP system. Load balancing is necessary only on systems where each processors has its own private queue of process which are eligible to execute.
- Load balancing is unncessary because once a processor becomes idle it immediately extracts a runnable process from the common run queue.
- On SMP, it is important to keep the workload balanced among all processors to fully utilize the benefits of having more than one processor else one or more processor will sit idle while other processors have high workloads along with lists of processors awaiting the CPU.

There are two general approaches to load balancing:

1. **Push Migration:** In push migration a task routinely checks the load on each processor and if it finds an imbalance then it evenly distributes

load on each processors by moving the processes from overloaded to idle or less busy processors.

2. **Pull Migration:** Pull Migration occurs when an idle processor pulls a waiting task from a busy processor for its execution.

Multicore Processors

- In multicore processors **multiple processor** core are placed on some physical chip.
- Each core has a register set to maintain its architectural state and thus appears to the operating system as a separate physical processor.
- **SMP systems** that use multicore processors are faster and consume less power than systems in which each processor has its own physical chip.
- However multicore processors may complicate the scheduling problems.
- When processor accesses memory then it spends a significant amount of time waiting for the data to become available, this situation is called **memory stall.** It occurs for various reasons such as cache miss, which is accessing the data that is not in the cache memory. In such cases the processor can spen upto fifty percent of its time waiting for data to become available from the memory. To solve this problem recent hardware designs have implemented multithreaded processor cores in which two or more hardware threads are assigned to each core. Therefor if one thread stalls while waiting for the memory, core can switch to another thread.

Process and Thread

- **Process:**

1. Processes are basically the programs that are dispatched from the ready state and are scheduled in the CPU for execution. PCB(Process Control Block) holds the concept of process. A process can create other processes which are known as Child Processes. The process takes more time to terminate and it is isolated means it does not share the memory with any other process.
2. The process can have the following states: new, ready, running, waiting, terminated, and suspended

- **Thread:**

1. Thread is the segment of a process which means a process can have multiple threads and these multiple threads are contained within a process.

2. A thread has three states: Ready, Running and Blocked.
3. The thread takes less time to terminate as compared to the process but unlike the process, threads do not isolate.

Difference between Process and Thread:

Multithreading in OS

- Multithreading is a technique used in operating systems to enable multiple threads of execution within a single process. A thread is a lightweight process that shares the same memory space and resources as other threads in the same process. Multithreading allows for concurrent execution of multiple tasks within a single process, making it possible to achieve better performance and responsiveness in certain types of applications
- Lets say, for example a program is not capable of reading keystrokes while making drawings. These tasks cannot be executed by the program at the same time. This problem can be solved through multi-tasking so that two or more tasks can be executed simultaneously.
- One of the benefits of multithreading is that it can improve overall system responsiveness by allowing applications to continue running even when some threads are blocked or waiting for I/O operations to complete. Multithreading can also help to reduce the overhead associated with context switching between processes, since switching between threads within a process is typically faster.
- The concept of multi-threading needs proper understanding of these two terms – a process and a thread.
  - A process is a program being executed. A process can be further divided into independent units known as threads.
  - A thread is like a small light-weight process within a process. Or we can say a collection of threads is what is known as a process.
- Multi-tasking is of two types:

0. Processor based: It is totally managed by the OS.
1. Thread based: It can be controlled by the programmer to some extent.

Applicaitons

Threading is a technique used in computer programming to improve the performance and responsiveness of software applications. It involves dividing a program into multiple smaller threads or tasks that can be executed independently and concurrently

- Threading is used widely in almost every field.
- Most widely it is seen over the internet nowadays where we are using transaction processing of every type like reacharges, online transfer, banking etc.
- GUI Applications: Threading is commonly used in Graphical User Interface (GUI) applications to keep the interface responsive while performing computationally intensive tasks. For example, when you click a button in a GUI application, a separate thread can be created to perform the task while the main thread keeps the interface responsive.
- Network Applications: Threading can be used in network applications to improve performance and scalability. For example, a server application can use multiple threads to handle incoming client connections simultaneously.
- Multimedia Applications: Threading is commonly used in multimedia applications to play audio and video files while simultaneously performing other tasks. For example, a media player can use a separate thread to play the media file while the main thread handles user input and interface updates.
- Parallel Processing: Threading can be used in parallel processing applications to speed up computations by dividing a large task into smaller independent tasks that can be executed simultaneously on different threads.
- Games: Threading can be used in games to keep the game running smoothly while performing tasks such as physics calculations, artificial intelligence, and rendering

Previous Year Questions

Q- What do you mean by PCB? Where is it used? What are its contents? Explain.

Q- Distinguish between the following:

- Process and Program
- Multiprogramming and multiprocessing
- Job scheduling and CPU scheduling

Q- Consider four process P1, P2, P3 and P4 with arrival time (0, 2, 4, 5) and burst time (7, 4, 1, 4) respectively, what is the average waiting time and turnaround time in SJF and Round Robin scheduling algorithm (with time quantum of 2ns) respectively?

Q- Draw the process state diagram and explain functionality of each state in detail.

Q- Explain the following:

1. Multithreading
2. Context switching
3. Schedulers

Assignment question answer

1. Some CPUs provide for more than two modes of operation. What are two possible uses of these multiple modes?

Ans. Multiple modes of operation in CPUs can be used for various purposes, including:

1. Privilege separation: Some CPUs provide multiple modes of operation to separate the privilege levels of software running on the system. This enables the system to provide different levels of access to resources and hardware devices based on the privilege level of the software. For example, operating systems can use different modes of operation to protect system resources and prevent unauthorized access to them.
2. Virtualization: Multiple modes of operation can also be used to enable virtualization, where multiple virtual machines (VMs) can run on a single physical machine. In this scenario, each VM can run in its own mode of operation, isolating it from other VMs and the host system. This provides increased security and flexibility, as each VM can run its own operating system and software stack, independently of other VMs on the same machine.

2.What are the three major activities of an operating system with regard to memory management?

Ans. The three major activities of an operating system with regard to memory management are:

1. Allocation: The operating system is responsible for allocating memory to different processes and applications running on the system. When a process or application requests memory, the operating system

determines whether there is enough free memory available and, if so, allocates a portion of memory to the process.

2. Deallocation: The operating system is also responsible for deallocating memory that is no longer needed by a process. When a process terminates or no longer requires a portion of memory that it had previously allocated, the operating system deallocates that memory and makes it available for other processes to use.
3. Protection: The operating system must ensure that each process can access only the memory that it has been allocated and prevent processes from accessing memory that belongs to other processes. The operating system uses techniques such as address translation and memory protection mechanisms to enforce memory protection and prevent processes from accessing unauthorized memory locations.

3. Provide two programming examples in which multi-threading does not provide better performance than a single-threaded solution

Ans. There are situations in which multithreading does not provide better performance than a single-threaded solution. Two programming examples are:

1. Small tasks: If the tasks that need to be performed are very small, the overhead of creating and managing multiple threads can actually outweigh the benefits of parallelization. For example, consider a program that needs to sort an array of 10 elements. In this case, creating multiple threads to sort the array would likely result in slower performance than simply sorting the array in a single thread.
2. Shared resources: If the threads in a multithreaded program need to access shared resources such as memory or files, managing access to these resources can become a bottleneck and limit the benefits of parallelization. For example, consider a program that reads data from a file, processes it, and writes the results to another file. If multiple threads are used to perform the processing, they may contend for access to the input and output files, leading to reduced performance compared to a single-threaded solution that processes the data sequentially.

In both of these examples, the overhead of creating and mananging can outweigh the benefits of parallelizaiton, leading to slower performance than a single-threaded solutio

Difference between dispatcher and scheduler

Scheduler and the Dispatcher are the two most important terms in an operating system. which are critical in the management of processes as well as the execution of processes**. While the scheduler chooses processes that must be run, the dispatcher is in charge of passing these processes to the CPU**. Despite the interrelation that might be observed when comparing the roles of these elements, they serve different direct purposes, which are aimed at optimizing CPU usage. A scheduler and a dispatcher are two important concepts that are used in operating systems when it comes to multitasking and executing processes.

What is a Scheduler?

Schedulers are a special type of operating system software that manages process scheduling in a variety of ways. Its main function is to select the jobs that are to be submitted to the system and decide which process will run.

There are three types of schedulers, which are as follows:

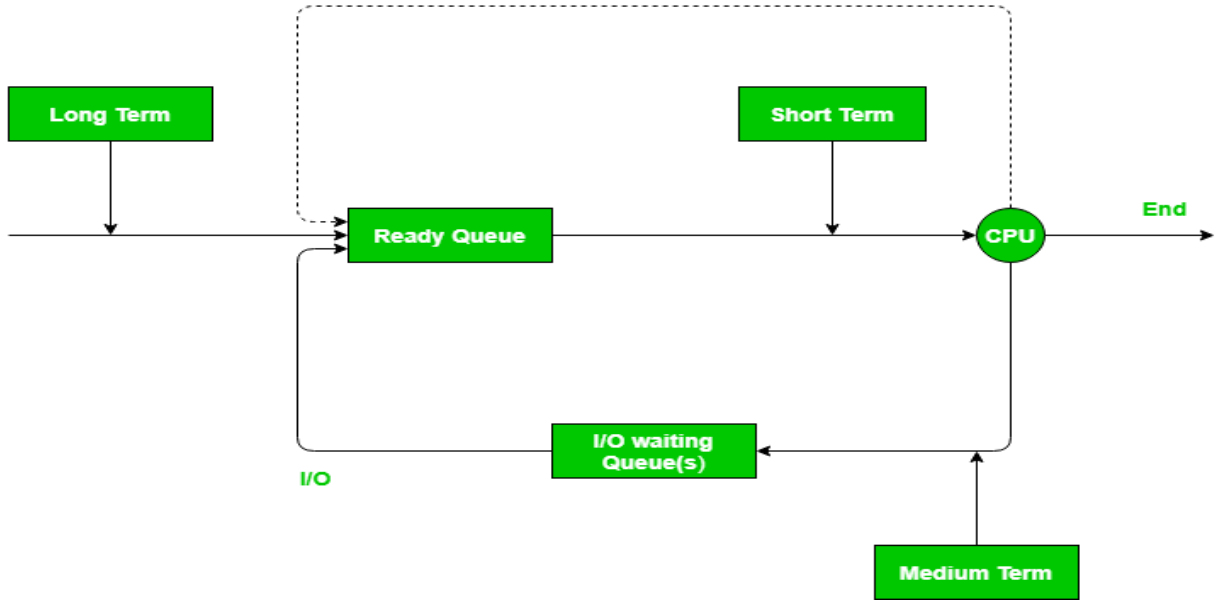
1. **Long-Term (job) Scheduler:** Due to the small size of the main memory, initially all the programs are stored in secondary memory. When they are loaded or stored in main memory, then they are known as processes. The long-term scheduler decides how many processes will remain in the ready queue. So, in simple words, the long-term scheduler decides the degree of multiprogramming of the system.
2. **Medium-Term Scheduler:** Often a running process needs an I/0 operation that does not require CPU time. That is why when a process needs I/O operation during its execution, the operating system sends that process to a blocked queue. When that process completes its I/O operation, then it is again shifted to the ready queue . All these decisions are taken by the medium-term scheduler. Medium-term scheduling is part of swapping.
3. **Short-Term (CPU) Scheduler:** When there are many processes initially in the main memory, all are present in the ready queue. Out of all these processes, only one is selected for execution. This decision is in the hands of the short-term scheduler or CPU scheduler.

Advantages of a Scheduler

- **Optimized CPU Utilization:** It helps to check that the CPU is loaded to the maximum possible extent by frequently choosing tasks to execute.
- **Fair Process Handling:** Demands a fixed amount of CPU time for each process like FCFS, SJF, and RR in order to offer equal opportunity for processes in the system.
- **Process Management:** Oversees the processes in different states (for instance, ready, blocked, or running).

Disadvantages of a Scheduler

- **Complexity:** They may also not be well suited for all system designs and can also be very hard to actually schedule.
- **Overhead:** There are some drawbacks in maintaining the scheduler, as the latter could present some overhead to the system, and this could be especially so in a real-time environment where decisions require to be made rapidly.



What is a Dispatcher?

Dispatcher is a special type of program whose work starts after the scheduler. When the scheduler completes its task of selecting a process, it is the dispatcher that moves the process to the queue it needs to go to.

The dispatcher is the module that hands over control of the CPU to the process that has been selected by the short-term scheduler.

The following things happen in this function:

1. **switching context:** Cores the current process and restores the state of the process to be run next.
2. **Switching to User Mode:** Makes sure that it runs in the user mode and not kernel mode, which is for security and privilege break-

3. **Jumps to the correct location in the user program** from where the program can be restarted.

Advantages of a Dispatcher

- **Fast Process Switching:** Evaluates circumstances where a procedure shifts from the ready queue to the execution phase in a way that causes less delay.
- **Efficient CPU Time Allocation:** Is important in making sure that processes receive CPU time, hence giving the necessary support for multitasking to occur.

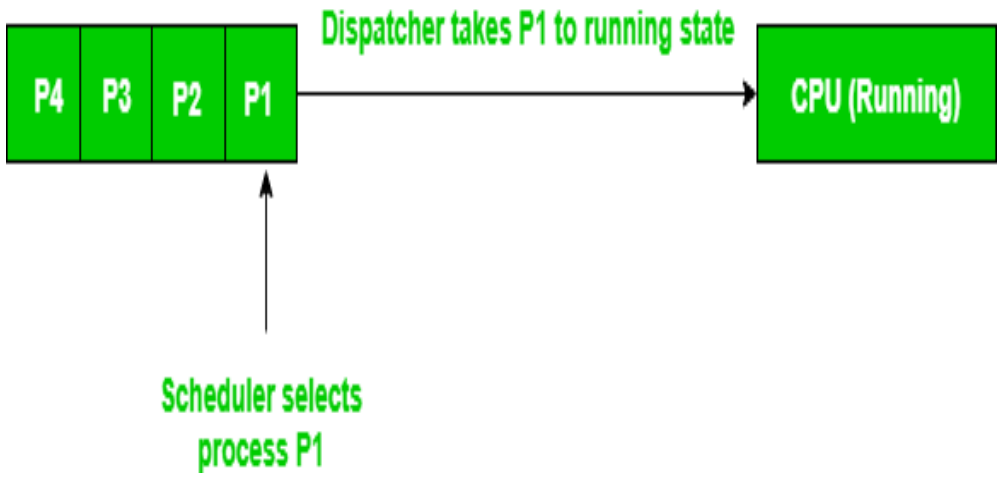
Disadvantages of a Dispatcher

- **Dispatch Latency:** Although the time taken is considerably small, this lateness in dispatching the requests can slow down the system.
- **Dependent on Scheduler:** The dispatcher hence cannot work on her own since she is reliant on the decisions made by the scheduler.

Difference Between Dispatcher and Scheduler

Consider a situation, where various processes are residing in the ready queue waiting to be executed. The CPU cannot execute all of these processes simultaneously, so the operating system has to choose a particular process on the basis of the scheduling algorithm used. So, this procedure of selecting a process among various processes is done by **the scheduler**. Once the scheduler has selected a process from the queue, the **dispatcher** comes into the picture, and it is the dispatcher who takes that process from the ready queue and moves it into the running state. Therefore, the scheduler gives the dispatcher an ordered list of processes which the dispatcher moves to the CPU over time.

Example – There are 4 processes in the ready queue, P1, P2, P3, P4; Their arrival times are t0, t1, t2, t3 respectively. A First in First out (FIFO) scheduling algorithm is used. Because P1 arrived first, the scheduler will decide it is the first process that should be executed, and the dispatcher will remove P1 from the ready queue and give it to the CPU.



The scheduler will then determine P2 to be the next process that should be executed, so when the dispatcher returns to the queue for a new process, it will take P2 and give it to the CPU. This continues in the same way for P3, and then P4.

Properties	DISPATCHER	SCHEDULER
Definition	Dispatcher is a module that gives control of CPU to the process selected by short term scheduler	Scheduler is something which selects a process among various processes
Types	There are no different types in dispatcher.It is just a code segment.	There are 3 types of scheduler i.e. Long-term, Short-term, Medium-term
Dependency	Working of dispatcher is dependent on scheduler.Means dispatcher have to wait until scheduler selects a process.	Scheduler works independently. It works immediately when needed
Algorithm	Dispatcher has no specific algorithm for its implementation	Scheduler works on various algorithm such as FCFS, SJF, RR etc.
Time Taken	The time taken by dispatcher is called dispatch latency.	Time taken by scheduler is usually negligible.Hence we neglect it.
Functions	Dispatcher is also responsible for:Context Switching, Switch to user mode, Jumping to proper location when process again restarted	The only work of scheduler is selection of processes.
Tasks	Dispatcher allocates the CPU to the process selected by the short-time scheduler.	Scheduler performs three task. Job scheduling (Long-term scheduler), CPU scheduling (Short-term scheduler) and swapping (Medium-term scheduler).

Properties	DISPATCHER	SCHEDULER
Purpose	To move the process from the ready queue to the CPU	To select the process and decide which process to run
Execution time	It takes a very short execution time	It takes longer execution time than dispatcher
Interaction	The dispatcher works with the CPU and the selected process	The scheduler works with the ready queue and the dispatcher

https://pravin-hub-rgb.github.io/BCA/resources/sem2/operating_sys/unit2/index.html